Domain-Specific Instruction Fine-Tuning: Comprehensive Study Guide & Code Masterclass

1. End-to-End Conceptual Roadmap



2. Theoretical Foundations: Why SFT is Essential

2.1 The Nature of a Base Model vs. Instruct Model

A base model is an **autocomplete engine**. It optimizes: $$\mathcal{L}_{\text{pretrain}}(\theta) = -\sum_{i=1}^{N} \log P(x_i \mid x_1, x_2, \dots, x_{i-1}; \theta)$$

When prompted with:

> *"Explain the mechanism of action of Metformin."*

A base model sees this as an incomplete sentence or a headline. It often predicts:

> *"...is a common examination question for second-year pharmacology students. Discuss its side effects and renal clearance."*

2.2 SFT (Supervised Fine-Tuning)

SFT forces the model to recognize a **conditioning boundary**:

- **Prompt context:** The task given to the model.
- **Target completion:** The authoritative response to generate.



3. Data Engineering for Instruction Tuning

3.1 Alpaca Format Breakdown

Field	Purpose	Example (Medical Domain)	When Empty?
`instruction`	The task or directive	*"Summarize clinical implications of drug interaction."*	Never
`input`	Contextual passage or reference data	*"Patient is taking Atorvastatin 40mg and Ezetimibe 10mg..."*	Set to `""` when the query is standalone
`output`	Ground-truth gold response	*"Combining Atorvastatin with Ezetimibe produces complementary LDL-C lowering..."*	Never

3.2 The Critical Flattening Step & Template Alignment

The model consumes a single 1D sequence of token IDs. The template string must be **strictly identical** between training and inference:

```
### Instruction:
Explain the mechanism of action of Metformin.

### Input:


### Response:
Metformin activates AMP-activated protein kinase (AMPK)... </s>
```

> **Golden Rule:** If training uses `### Response:\n` and inference passes a raw question without `### Response:\n`, generation degrades into incoherent rambling or prompt repetitions.

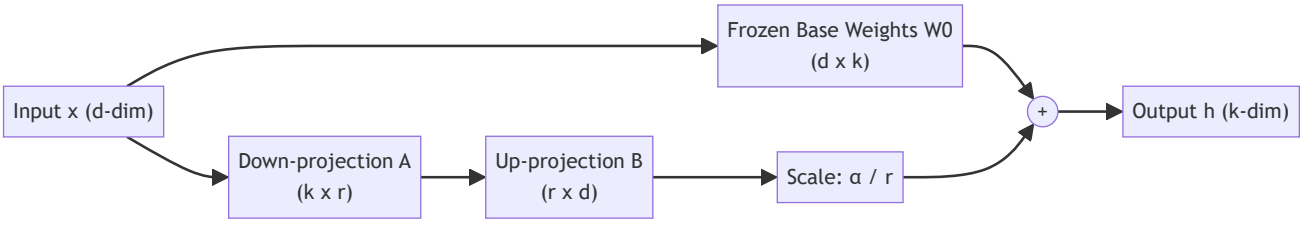
---

4. Parameter-Efficient Fine-Tuning: LoRA & QLoRA

4.1 The Mathematics of LoRA

Instead of updating the full weight matrix $W_0 \in \mathbb{R}^{d \times k}$ (which requires billions of floating-point numbers in optimizer memory): $$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} (B \cdot A)$$

- W_0 is **frozen** (requires 0 gradient memory).
- $A \in \mathbb{R}^{r \times k}$ initialized from $\mathcal{N}(0, \sigma^2)$.
- $B \in \mathbb{R}^{d \times r}$ initialized to 0 (so $\Delta W = 0$ at step 0).
- $r \ll \min(d, k)$ is the low-rank dimension (typically 8 to 64).



4.2 LoRA Hyperparameters Decoded

Hyperparameter	Meaning	Typical Range	Recommended for TinyLlama (1.1B)	Impact
`r` (Rank)	Latent dimensionality of adapter matrices	4 – 64	`r = 8` or `16`	Higher r captures subtle domain semantics but uses more VRAM.

Hyperparameter	Meaning	Typical Range	Recommended for TinyLlama (1.1B)	Impact
`lora_alpha`	Scaling multiplier ($\Delta W \times \frac{\alpha}{r}$)	$1 \times r$ to $2 \times r$	`16` (when $r=8$)	Higher alpha increases the weight of learned adaptations relative to base knowledge.
`lora_dropout`	Dropout applied to adapter outputs	0.0 – 0.1	`0.05`	Prevents overfitting on small domain datasets.
`target_modules`	Linear layers receiving LoRA matrices	Attention / MLP	`["q_proj", "v_proj"]` or `["q_proj", "k_proj", "v_proj", "o_proj"]`	Tuning only Q, V minimizes parameter count; adding MLP layers improves reasoning.
`bias`	Train bias terms	`"none"`, `"all"`, `"lora_only"`	`"none"`	Avoids saving unneeded parameters.

---

5. Detailed Notebook Walkthrough: Cell-by-Cell Analysis

Phase 1: Baseline Verification (Cells 1 to 11)

- **Cell 1–4**: Loads base tokenizer and intermediate TinyLlama checkpoint.
  - Setting `tokenizer.pad_token = tokenizer.eos_token` prevents crashes during batching.
- **Cell 5–7**: Extracts and loads `/content/checkpoint-5` (a stage-1 domain model).
  - *Key Concept*: `device_map="auto"` automatically distributes layers across available CUDA VRAM.
- **Cell 8–11**: Tests text continuation.
  - Generates text from `"Clinical trials demonstrated that combining Atorvastatin with Ezetimibe"`.
  - Confirms the model continues medical sentences but does not act as an assistant.

Phase 2: Dataset Formatting Exploration (Cells 16 to 27)

- **Cell 16–22**: Uses `Amod/mental_health_counseling_conversations`.
  - Illustrates difference between `[INST] ... [/INST]` (Llama-2 chat style) and custom headers.
- **Cell 23–27**: Converts Hugging Face `Dataset` to Pandas DataFrame, saves to `.csv` and `.jsonl`, then reloads using `load_dataset("csv", ...)`. Demonstrates loading local data.

Phase 3: Domain Instruction Tuning Setup (Cells 30 to 38)

- **Cell 30–35**: Loads `pharma_instruction_data.csv` and maps `format_example()`.
  - Constructs `### Instruction:\n...\n### Input:\n...\n### Response:\n...`.
- **Cell 37–38**: Tokenization function.
  - `tokens["labels"] = tokens["input_ids"].copy()`: Standard causal LM setting where labels match input IDs.

Phase 4: LoRA Configuration & Training (Cells 39 to 46)

- **Cell 40–42**: Creates `LoraConfig` and wraps base model with `get_peft_model()`.
- **Cell 44–46**: Sets up `TrainingArguments` and `Trainer`.
  - `per_device_train_batch_size=1` with `gradient_accumulation_steps=8` gives an effective batch size of 8 without OOM.
  - `fp16=True` uses half-precision arithmetic for 2x faster execution and 50% memory reduction.

Phase 5: Verification & Generation (Cells 47 to 57)

- **Cell 47–53**: Reloads trained checkpoint and performs generation.
- **Cell 55**: Complete script with response masking and test prompts.
- **Cell 56–57**: Direct head-to-head comparison between Non-Instruction model and Instruction-Tuned model on medical queries.

---

6. Critical Bugs & Deficiencies in the Notebook (And How to Fix Them)

Bug 1: Missing Response Loss Masking in Cell 37

- **The Issue**: In Cell 37:

  ```
  tokens["labels"] = tokens["input_ids"].copy()
  ```

  The loss is computed over the entire sequence, including `### Instruction: ...`.
- **The Consequence**: The model wastes capacity learning to predict the user prompt instead of learning solely how to answer.
- **The Solution**: Set label token IDs prior to `### Response:` to `-100` (PyTorch CrossEntropyLoss ignores `-100`).

Bug 2: Substring Tokenizer Index Bug in Cell 55

- **The Issue**:

  ```
  response_token_start = len(tokenizer(text[:response_start])["input_ids"])
  ```

  Tokenizing substrings independently produces incorrect token counts because subword tokenizers handle prepended whitespaces and BOS tokens differently when string prefixes change!
- **The Solution**: Tokenize the full string once, then scan the tokenized IDs for the response marker sequence or use Hugging Face TRL's `DataCollatorForCompletionOnlyLM`.

Bug 3: Missing EOS Token at the End of Prompt

- **The Issue**:

  ```
  prompt = f"### Instruction:\n{example['instruction']}\n### Input:\n{example['input']}\n###
  Response:\n{example['output']}"
  ```

  No `tokenizer.eos_token` (`</s>`) is appended!
- **The Consequence**: The model never learns when to stop speaking and repeats gibberish until reaching `max_new_tokens`.
- **The Solution**:

  ```
  prompt = f"### Instruction:\n{example['instruction']}\n### Input:\n{example['input']}\n###
  Response:\n{example['output']}{tokenizer.eos_token}"
  ```

Bug 4: Incorrect LoRA Loading at Inference (Cell 48)

- **The Issue**:

  ```
  model_path = "/content/tinyllama-instruction/checkpoint-3"
  instruction_model = AutoModelForCausalLM.from_pretrained(model_path, device_map="auto")
  ```

  `Trainer.save_model` with PEFT saves only adapter files (`adapter_config.json`, `adapter_model.safetensors`). Calling `AutoModelForCausalLM.from_pretrained` on an adapter folder will fail or load an un-adapted model.
- **The Solution**:

  ```
  from peft import PeftModel
  base_model = AutoModelForCausalLM.from_pretrained(base_model_path, device_map="auto")
  instruction_model = PeftModel.from_pretrained(base_model, model_path)
  ```

Bug 5: NameError in Cell 45

- **The Issue**: Cell 42 creates `instruction_model = get_peft_model(...)`, but Cell 45 passes `model=instruction_model_lora`. This throws `NameError: name 'instruction_model_lora' is not defined`.
- **The Solution**: Align variable names consistently to `instruction_model`.

---

7. Production-Grade Reference Code

```
import torch
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    TrainingArguments,
    Trainer,
    DataCollatorForSeq2Seq
)
from peft import LoraConfig, get_peft_model, TaskType, PeftModel

# 1. Base Model & Tokenizer
MODEL_ID = "TinyLlama/TinyLlama-1.1B-intermediate-step-1431k-3T"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# 2. Template Formatter with EOS
def format_alpaca(example):
    inp = example["input"].strip() if example.get("input") else ""
    if inp:
        text = f"### Instruction:\n{example['instruction']}\n\n### Input:\n{inp}\n\n###
Response:\n{example['output']}{tokenizer.eos_token}"
    else:
        text = f"### Instruction:\n{example['instruction']}\n\n### Response:\n{example['output']}
{tokenizer.eos_token}"
    return {"text": text}

# 3. Robust Tokenization with Response Loss Masking
def tokenize_and_mask_prompt(example):
    full_tokens = tokenizer(
        example["text"],
        truncation=True,
        max_length=512,
        padding=False # Collator will pad dynamically
    )

    # Locate response marker token sequence
    marker = "### Response:\n"
    marker_ids = tokenizer.encode(marker, add_special_tokens=False)

    input_ids = full_tokens["input_ids"]
    labels = list(input_ids)

    # Find start index of response marker in input_ids
    marker_len = len(marker_ids)
    response_idx = -1
    for i in range(len(input_ids) - marker_len + 1):
        if input_ids[i:i+marker_len] == marker_ids:
            response_idx = i + marker_len
```

```
            break

    if response_idx != -1:
        # Mask prompt tokens with -100
        labels[:response_idx] = [-100] * response_idx
    else:
        # Fallback: if not found, do not calculate loss
        labels = [-100] * len(labels)

    full_tokens["labels"] = labels
    return full_tokens

# 4. LoRA Setup
lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
    bias="none"
)

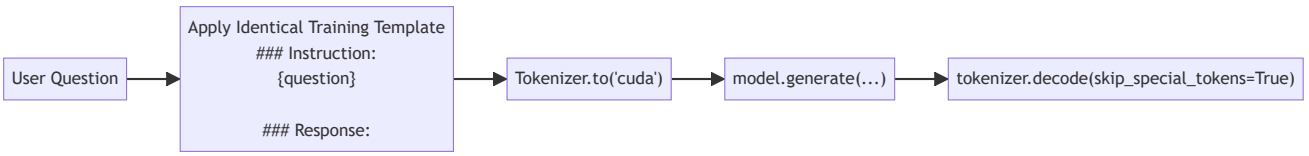
base_model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.float16,
    device_map="auto"
)
model = get_peft_model(base_model, lora_config)
model.print_trainable_parameters()

# 5. Training Arguments
training_args = TrainingArguments(
    output_dir="./tinyllama-pharma-instruct",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    warmup_ratio=0.03,
    fp16=True,
    logging_steps=10,
    save_strategy="epoch",
    evaluation_strategy="no",
    report_to="none"
)

# 6. Dynamic Padding Collator
collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,
    model=model,
    padding=True,
    label_pad_token_id=-100
)
```

---

8. Generation & Inference Mechanics



Generation Hyperparameters

Parameter	Purpose	Typical Value	Medical Domain Recommendation
`temperature`	Softmax entropy scaling ($P_i = \frac{e^{z_i/T}}{\sum e^{z_j/T}}$)	0.1 – 1.0	**0.2 – 0.4**: Lower temperature prevents pharmacological hallucinations.
`top_p` (Nucleus)	Considers smallest set of tokens whose cumulative probability $\ge p$	0.8 – 0.95	**0.85 – 0.9**: Filters out low-probability nonsense tokens.
`repetition_penalty`	Multiplier penalizing previously generated token logits	1.05 – 1.2	**1.1 – 1.15**: Prevents repetitive looping on chemical nomenclature.
`do_sample`	Greedily pick top token (`False`) vs stochastic sampling (`True`)	Boolean	`False` for deterministic answers, `True` for varied prose.

---

9. Comprehensive Comparison Matrix

Dimension	Pretraining	Non-Instruction Domain FT	Instruction SFT	RAG
Goal	General language synthesis	Learn specialized vocabulary & style	Learn to follow directives	Retrieve external ground-truth facts
Data Format	Raw internet text (unstructured)	Unstructured domain papers (PDF/PubMed)	(Instruction, Input, Output) structured	Chunks in vector DB + metadata
Loss Target	All tokens	All tokens	**Response tokens only** (masked prompt)	None (Frozen model)
Computational Cost	Millions of USD	Moderate (GPUs for days)	Low (Single GPU for hours with LoRA)	Zero model training cost
Handles Fresh Facts?	No (knowledge cutoff)	No (slow update cycle)	No	**Yes** (real-time DB query)
Behavioral Control	None (autocomplete)	None (domain autocomplete)	**High** (task, style, structure)	High (prompt-conditioned)

---

10. Exam & Interview Quick-Review Checklist

1. **Why does base model generation fail on prompts?** Base models model the distribution $P(\text{next token} \mid \text{previous tokens})$. They have no concept of user vs. assistant; a question is continued as text rather than answered.
2. **What happens if you do not mask prompt tokens in SFT?** The loss function penalizes errors made predicting the prompt itself, leading to suboptimal adapter updates and possible prompt regurgitation.
3. **Why must the training template match inference?** The attention layers learn positional and semantic trigger patterns from markers like `### Response:\n`. Missing those triggers leaves the model in base continuation mode.
4. **Why is `eos_token` essential in training data?** The model learns when to cease generation by predicting the EOS token. Without it, generation continues until reaching `max_new_tokens`.
5. **What is the difference between LoRA rank (r) and alpha (α)?** r defines the width of the adapter bottleneck (capacity); α scales the gradient update magnitude ($\frac{\alpha}{r}$).